# Stylized Weather Documentation

[Online Documentation]

## StylizedWeatherParticleWrapper.cs

Keeps particle systems continuously wrapping around a bounded area, giving the illusion of endless weather effects such as rain, snow, or dust that follow the player or camera.

---

### How It Works

1. **Initialization (`Start`)**
    - Caches required references (particle system, controller, and follow object).
    - Allocates an internal particle buffer based on the system's maximum particles.
    - Calculates the wrapping limits (`xLimit`, `zLimit`) from the particle system's shape size.
    - Sets a vertical offset so the system correctly follows its target (e.g., player or camera).
2. **Continuous Update (`Update`)**
    - Repositions the entire particle system to follow the assigned `followObject`, maintaining a consistent offset in height.
    - This ensures the particle area always stays centered around the player or scene camera.
3. **Wrapping Loop (`WrapParticles`)**
    - Runs at an interval defined by `updateDelay` (0.01–0.5 seconds).
    - Iterates through all alive particles and checks whether each one has exited the wrapping bounds along the X or Z axis.
    - If a particle is outside:
        - It is repositioned to the opposite side of the system's area.
        - Optionally, its lifetime and/or velocity can be modified.
    - Updated particles are then written back into the particle system.

4. **Gizmo Visualization (Editor Only)**
    - Draws red box outlines in the Scene view to show the current wrapping bounds.
    - Helps designers visualize the area where particles loop around.

---

## Inspector

| Field | Description |
|---|---|
| **Wrap Particles** (`wrapParticles`) | Enables or disables the wrapping behavior. If disabled, the system simply follows the target without looping particles. |
| **Update Delay** (`updateDelay`) | How frequently the script checks and wraps particles. Lower values give smoother transitions but use slightly more CPU. |
| **Change Lifetime After Wrap** (`changeLifeTimeAfterWrap`) | If enabled, the lifetime of wrapped particles is reduced proportionally to `lifetimeAfterWrap`. |
| **Lifetime After Wrap** (`lifetimeAfterWrap`) | Multiplier applied to a particle's original lifetime after wrapping (0.0–1.0). |
| **Change Velocity After Wrap** (`changeVelocityAfterWrap`) | If enabled, reduces a particle's velocity after wrapping. |
| **Velocity Multiplier After Wrap** (`velocityMultiplierAfterWrap`) | Multiplier applied to the particle's velocity upon wrapping (0.0–1.0). |

# *StylizedWeatherController.cs*

Central manager that controls all weather systems, particle effects, and environment behavior (e.g., fog, wind, and visual transitions) for the Stylized Weather System.

---

## Overview

The `StylizedWeatherController` acts as the **main coordinator** for multiple weather types and their particle effects. It controls when certain weather presets are active, manages their transitions, synchronizes particle systems, and ensures they follow a specific target such as a camera or player.

Each weather preset is defined by a `StylizedWeatherSettings` component, which holds configuration data (particle effects, fog, lighting, etc.) for that weather type.

---

## How It Works

1. **Initialization**
    - On startup, the controller detects all `StylizedWeatherSettings` and particle systems inside the assigned `particleContainer`.
    - It optionally activates the default weather setup.

2. **Weather Control**

- `ChangeWeather(string name)` activates a weather preset based on its GameObject name.
- `DisableAll(StylizedWeatherSettings enabledW, bool quickly)` disables all other weather presets and can optionally fade out existing particle effects faster.

3. **Particle System Management**
   - The controller maintains a reference to all weather particle systems (`allParticles`) for fast access.
   - When switching or disabling weather, it reduces emission rates and optionally shortens particle lifetimes to quickly clear old effects.

4. **Editor Tools**
   - The controller automatically updates its particle list when gizmos are enabled.
   - In the Inspector, all detected weather presets appear as clickable buttons for quick switching.
   - Displays helpful messages to guide setup and ensure gizmos are active during editing.

---

## Inspector

| Field | Description |
|---|---|
| **followObject** | The transform (e.g., camera or player) the weather system should follow. |
| **affectGlobalFog** | Toggles whether weather effects modify the global fog settings. |
| **drawGizmos** | Enables drawing of gizmos for debugging and scene visualization. |
| **defaultWeather** | The default weather GameObject to activate on startup. |
| *(Hidden Fields)* | `weatherSettings`, `particleContainer`, `allParticles`, `windZone` are automatically managed references. |

## *StylizedWeatherSettings.cs*

Defines a single weather preset — controlling its particle systems, environmental fog, and wind behavior — for use with the `StylizedWeatherController`.

---

## Overview

Each `StylizedWeatherSettings` component represents one **weather state** (for example, *Rain*, *Snow*, *Fog*, *Sunny*, etc.).
It holds references to all related particle systems and defines how they should behave — including emission rate, lifetime, speed, color, size, and external forces.

When activated, this script:

- Disables all other weather presets managed by the controller.
- Updates all assigned particle systems to match the defined settings.
- Optionally adjusts global fog color and density for environmental blending.
- Syncs wind strength with the scene's `WindZone`.

## How It Works

1. **Activation (`OnEnable`)**
   - When the preset is enabled:
     - Disables other weather presets through `StylizedWeatherController.DisableAll()`.
     - Updates the wind strength (`WindZone.windMain`) based on `windPower`.
     - Calls `UpdateSystems()` to apply all particle settings.
     - Starts a gradual fog color and density update (if enabled in the controller).

2. **Updating Fog (`StartUpdatingGlobalFog` & `UpdateGlobalFog`)**
   - Smoothly transitions the scene's fog density and color toward the defined values.
   - Uses `Mathf.Lerp` and `Color.Lerp` over time until the fog matches the target values exactly.
   - Updates every 0.1 seconds until the target state is reached.

3. **Updating Particle Systems (`UpdateSystems`)**
   - Runs every 0.5 seconds to keep all particle settings synchronized.
   - Iterates through each `Element` and applies its properties to the corresponding `ParticleSystem`.
   - Updates emission, size, color, lifetime, speed, external forces, and collision state.

4. **Editor Features**
   - The custom editor validates the configuration:
     - Automatically fills each element's name to match its assigned particle system.
     - Warns about duplicate particle system assignments.
     - Ensures that collision settings are only used for correctly configured particle systems.

## The `Element` Struct

Each entry in the `elements` array represents an individual particle system used in this weather preset.

| Field | Description |
|---|---|
| **name** | Automatically filled with the particle system's name for clarity. |
| **particleSystem** | Reference to the ParticleSystem to control. |
| **size** | Start size curve for the particle system. |
| **color** | Start color gradient or color range. |
| **lifetime** | Particle lifetime curve. |
| **emission** | Emission rate over time curve. |
| **speed** | Start speed curve. |
| **externalForce** | Curve applied to the external force module multiplier. |
| **enableCollision** | Enables or disables particle collision for this system. |

## Main Inspector

| Field | Description |
|---|---|
| **windPower** | Strength of the wind (applied to the controller's WindZone). |
| **globalFogDensity** | Target fog density value for this weather preset. |

| | |
|---|---|
| **globalFogColor** | Target fog color for this weather preset. |
| **quickDisable** | If true, old weather effects fade out faster when switching. |
| **hotkey** | Optional keyboard key for quickly toggling or testing this weather preset. |
| **elements** | List of particle system elements that define this weather preset. |

---

## Typical Use Case

Each `StylizedWeatherSettings` instance should be attached to a child GameObject of your main `StylizedWeatherController`.
For example:

StylizedWeatherController

├── Rain (StylizedWeatherSettings)

├── Snow (StylizedWeatherSettings)

├── Fog (StylizedWeatherSettings)

When `Rain` is activated:

- Other weather presets are disabled.
- The rain particle systems' speed, lifetime, and color are applied.
- The fog gradually adjusts to the rainy fog density and color.
- The wind zone strength changes to match the rain's wind power.

# *StylizedWeatherTriggerZone.cs*

Automatically triggers a weather preset when a player enters or exits a defined 3D zone, integrating with the `StylizedWeatherController`.

---

## Overview

`StylizedWeatherTriggerZone` uses a **BoxCollider** set as a trigger to detect when an object (typically the player) enters or exits the zone.
When triggered, it activates a specific weather preset on enter and optionally another on exit.
This allows for dynamic weather changes in the scene based on location, such as:

- Entering a rainy forest → triggers rain.
- Leaving a cave → reverts to sunny weather.
- Walking through multiple zones → layered weather transitions.

---

## How It Works

1. **Setup (`Reset` & `Start`)**
   - Ensures the attached BoxCollider is configured as a trigger.
   - Locates the active `StylizedWeatherController` in the scene.

2. **Trigger Detection**
   - `OnTriggerEnter(Collider other)`

        - Checks if the collider matches the optional `playerTag`.
        - If allowed, calls `ChangeWeather(enterWeatherName)` on the controller.
        - Marks the trigger as used if `reusable` is false.
    - `OnTriggerExit(Collider other)`
        - Checks if exit handling is allowed.
        - Calls `ChangeWeather(exitWeatherName)` if defined.
        - Resets `_hasTriggered` if the trigger is reusable.

3. **Editor Gizmos**
    - Draws a semi-transparent cube in the Scene view representing the trigger area.
    - Draws a wireframe outline for clarity.
    - Displays a label with the enter and exit weather names for easy identification.

---

## Inspector Fields

| Field | Description |
|---|---|
| **enterWeatherName** | Name of the weather preset to activate on entering the trigger. |
| **exitWeatherName** | Name of the weather preset to activate on leaving the trigger. |
| **playerTag** | Optional tag for detecting the player. Default: `"Player"`. |
| **reusable** | If true, the trigger can be used multiple times. |
| **showGizmos** | Toggle visibility of the trigger in the Scene view for debugging. |

## *StylizedWeatherWindZone.cs*

Dynamically rotates a GameObject (usually a WindZone) over time to simulate changing wind direction in a stylized weather system.

---

## Overview

`StylizedWeatherWindZone` randomly changes the yaw (Y-axis rotation) of a GameObject within a defined range and interval, optionally using an easing curve for smooth transitions.
This is typically used with Unity's `WindZone` component to create natural, dynamic wind behavior for weather effects like trees, particles, or environmental foliage.

---

## How It Works

1. **Initialization (`Start`)**
    - Validates input ranges (`minInterval`, `maxInterval`, `rotationDuration`).
    - Schedules the first interval.
    - If `startImmediate` is true, immediately picks a new target rotation.

2. **Rotation Logic (`Update`)**
    - **Waiting Interval:** When not rotating, the script counts `intervalTimer` until the next rotation is triggered.
    - **Rotating:** While rotating:

        - Interpolates from `startRot` to `targetRot` over `rotationDuration`.
        - If `useEasingCurve` is enabled, applies the curve for smooth acceleration/deceleration.
        - Ends rotation once the duration is complete and schedules the next interval according to `intervalAfterRotation`.

3. **Target Selection (`PickNewTarget`)**
    - Randomly chooses a new yaw offset between `minYawChange` and `maxYawChange`.
    - Updates `targetRot` and starts the rotation timer.

4. **Interval Scheduling (`ScheduleNextInterval`)**
    - Picks a random duration between `minInterval` and `maxInterval` before the next rotation.
    - Resets the interval timer.

---

## Inspector Fields

| Field | Description |
|---|---|
| **minInterval / maxInterval** | Minimum and maximum wait times between rotations. |
| **minYawChange / maxYawChange** | Range of yaw rotation change (degrees) per rotation. |
| **rotationDuration** | Time it takes to rotate from current to target yaw. |
| **useEasingCurve** | If true, applies `easingCurve` to smooth rotation over time. |
| **easingCurve** | AnimationCurve controlling rotation smoothing. |
| **startImmediate** | If true, performs a rotation immediately on start. |
| **intervalAfterRotation** | If true, waits for the next interval after completing rotation; otherwise waits concurrently. |